

M04 TEACHING BOOK

Video first -> close video -> self-contained solo teaching | 16 lessons

VIDEO FIRST -> CLOSE VIDEO -> SOLO TEACHING BOOK -> GUIDED REPRODUCTION -> INDEPENDENT TASK -> VALIDATE -> EXPLAIN IN YOUR OWN WORDS -> PROTECTED REVIEW ONLY AFTER A SAVED ATTEMPT -> FRESH CHANGED RETRY IF WEAK -> EVIDENCE / MASTERY

Use one lesson at a time. A video is a visual teacher, not the source of truth. After watching the mapped segment, close it and learn/solve from this book. Every lesson below contains enough explanation, a worked pattern, guided reproduction, validation, recovery and a changed independent transfer route to remain usable without the video.

DE04-01 - Python setup and first program

Source route: Luke Barousse - Python for Data Analytics | 0:08:42-0:18:52 (Getting Started) | [OPEN SOURCE](#)
Independent web recheck 2026-09-10: video/title/chapter map confirmed.

Starting state / prerequisite: M03 complete. No WSL or Git required. Install/use Python only because M04 is the first Python module.

1. Outcome

Run a Python program from a clean start, distinguish code from output, and use an error message as evidence instead of guessing.

2. Explain - beginner-first model

Python is an interpreter: it reads valid Python instructions and evaluates them according to the language rules. A program can be one line or many files. For a beginner, the safest loop is tiny change -> run -> inspect output/error -> explain what happened.

For M04 on Windows, use the current stable Python 3 release. The course baseline checked on 2026-09-10 is Python 3.14.7. You do not need WSL, Git, Docker, Airflow or PostgreSQL for this module. A browser notebook can be used for the first basic exercises, but by DE04-08 you need a normal folder with .py files so imports and reproducible scripts are real rather than simulated.

3. Demonstrate / inspect

```
print("Hello, data engineering")
print(2 + 3)
# Change one value, run again, and compare the output.
```

4. Guided reproduction

Run both lines. Change the message. Create one simple syntax error on purpose, read the exception/line indicator, repair only that defect, then run again from a clean start.

5. Validate the guided result

Expected output appears; the error is preserved in your notes; the repaired file runs after closing and reopening the interpreter/editor.

6. Recover when stuck

If nothing runs, first prove which Python executable you are using (`py --version` or `python --version`). If syntax fails, read the last error line and inspect quotes, parentheses and indentation near the reported location.

7. Independent task - lesson/video closed

With the video closed, create `p01\_first\_program.py` that prints your name, `Standalone DE 2026`, and the result of $7 * 8$. Create and repair one syntax error, preserving the first failed output.

8. Explain in your own words

What does Python execute, what is output, and how did the error message tell you where to inspect?

9. Protected review / fresh retry rule

Save the genuine independent attempt first. If weak, open only the matching protected Review section. Repair, close it, then do the changed retest without copying.

Changed retest if assigned: Create a fresh file that prints a different label and calculates $9 * 6$. Introduce a different syntax error from your first attempt, explain it, then repair it without opening Review R01.

10. Evidence / mastery

p01_first_program.py + first failed output + repaired output + 3-5 sentence explain-back.

Mastery: changed task passes validation + explain-back. Work corrected with Review open is repair evidence, not mastery.

Practice files: M04_PYTHON_PRACTICE_CANONICAL.py

DE04-02 - Variables, data types and operators

Source route: Luke Barousse - Python for Data Analytics | 0:18:52-0:48:41 (Variables, terms, data types) | [OPEN SOURCE](#)
Independent web recheck 2026-09-10: chapter timestamps confirmed.

Starting state / prerequisite: DE04-01 mastered or explicitly remediated.

1. Outcome

Represent identifiers, counts, money-like values, status text, booleans and missing values with intentional Python types and valid operators.

2. Explain - beginner-first model

A variable is a name bound to a value. The value has a type, and the type affects which operations are meaningful. `4` is an integer; `850.0` is a floating-point number; `"O2001"` is text even though it contains digits; `True`/`False` are booleans; and `None` represents the absence of a Python value.

Do not confuse data meaning with appearance. An order ID should usually be text because you do not add or average IDs. Arithmetic operators compute values; comparison operators produce booleans. Use `type(value)` while learning to check what Python actually created.

3. Demonstrate / inspect

```
order_id = "O2001"
qty = 4
unit_price = 850.0
gross = qty * unit_price
is_large = gross >= 3000
print(order_id, gross, is_large, type(gross))
```

4. Guided reproduction

Create examples of int, float, str, bool and None. Print each value and its type. Then calculate `qty * unit_price` and one boolean comparison.

5. Validate the guided result

Types match business meaning; arithmetic is applied only to measures; comparison output is boolean; you can explain why an identifier remains text.

6. Recover when stuck

Print the value and `type(value)` before changing code. If concatenation/arithmetic fails, inspect whether text needs parsing or whether the field should remain text.

7. Independent task - lesson/video closed

Create `order_id="O3107"`, `qty=4`, `unit_price=850.0`, `status="Completed"`. Calculate gross sales and a boolean for `gross >= 3000`. Explain the type of every variable.

8. Explain in your own words

Why are identifiers often strings, and what is the difference between `=` and `==`?

9. Protected review / fresh retry rule

Save the genuine independent attempt first. If weak, open only the matching protected Review section. Repair, close it, then do the changed retest without copying.

Changed retest if assigned: Use `shipment_id="S009"`, `cartons=7`, `unit_cost=425.5`, `delivered=False`. Calculate cost and `needs_review = (cost > 2500 and not delivered)`.

10. Evidence / mastery

Saved code + printed values/types + explanation of identifier vs measure.

Mastery: changed task passes validation + explain-back. Work corrected with Review open is repair evidence, not mastery.

Practice files: M04_PYTHON_PRACTICE_CANONICAL.py

DE04-03 - Strings and formatting

Source route: Luke Barousse - Python for Data Analytics | 0:48:41-1:15:10 (Strings, formatting, operators lead-in) | [OPEN SOURCE](#)
Independent web recheck 2026-09-10: chapter timestamps confirmed.

Starting state / prerequisite: DE04-02.

1. Outcome

Clean whitespace/case deliberately, build readable labels with f-strings, and preserve the original when normalization might alter business meaning.

2. Explain - beginner-first model

Strings are sequences of characters. Common cleaning methods such as `.strip()`, `.lower()`, `.upper()` and `.title()` return new strings; they do not magically define the correct business value. Cleaning should therefore be deliberate and auditable.

F-strings make labels readable by placing expressions inside `{}`. Use `repr(raw_text)` when invisible whitespace matters because `repr` exposes leading/trailing spaces and escape characters.

3. Demonstrate / inspect

```
raw = "  NORTH zone  "  
clean = raw.strip().title()  
print(repr(raw))  
print(f"Region: {clean}")
```

4. Guided reproduction

Apply strip/lower/upper/title to sample labels and compare `repr` before/after. Build two f-string labels.

5. Validate the guided result

Whitespace changes are visible; casing is intentional; original can still be compared; f-string output matches the required label exactly.

6. Recover when stuck

If output looks identical, print `repr(value)`. If title-case changes a name incorrectly, stop using it as a blanket business rule and define an explicit mapping instead.

7. Independent task - lesson/video closed

Normalize " EAST hub " to "East Hub" and create `Hub: East Hub | Status: Ready`. Preserve and print the raw value too.

8. Explain in your own words

Why is string cleaning not the same as business standardization?

9. Protected review / fresh retry rule

Save the genuine independent attempt first. If weak, open only the matching protected Review section. Repair, close it, then do the changed retest without copying.

Changed retest if assigned: Normalize " west DEPOT " to an agreed label and create a separate message containing the original and normalized values.

10. Evidence / mastery

Raw + normalized output + code + one caveat about automatic casing.

Mastery: changed task passes validation + explain-back. Work corrected with Review open is repair evidence, not mastery.

Practice files: M04_PYTHON_PRACTICE_CANONICAL.py

DE04-04 - Conditions and boolean logic

Source route: Luke Barousse - Python for Data Analytics | 1:26:20-1:34:24 (Conditional statements) | [OPEN SOURCE](#)
Independent web recheck 2026-09-10: chapter timestamp confirmed.

Starting state / prerequisite: DE04-03.

1. Outcome

Translate a business rule into if/elif/else with correct AND/OR behavior and boundary tests.

2. Explain - beginner-first model

A condition is an expression whose result is truthy/falsey; explicit comparisons usually make business rules easier to audit. `and` requires both sides to be true, while `or` requires at least one. Parentheses are useful when a rule combines several conditions.

Boundary tests are part of the code, not optional decoration. If the rule says amount $\geq$ 2500, test just below, exactly at, and above 2500, plus a status that should disqualify the row.

3. Demonstrate / inspect

```
status = "Completed"
amount = 2500
if status == "Completed" and amount >= 2500:
    label = "priority"
else:
    label = "standard"
print(label)
```

4. Guided reproduction

Test Completed/2800, Completed/2499, Completed/2500 and Pending/3000. State the expected label before running.

5. Validate the guided result

Boundary cases behave exactly as the written rule; a high Pending amount does not become priority.

6. Recover when stuck

Write the English rule in one sentence, circle the words AND/OR, then translate one clause at a time. Print intermediate booleans if needed.

7. Independent task - lesson/video closed

Classify an order as `priority` only if status is Completed and amount $\geq$ 2500; otherwise `standard`. Use at least four boundary cases.

8. Explain in your own words

When would replacing AND with OR change the eligible rows?

9. Protected review / fresh retry rule

Save the genuine independent attempt first. If weak, open only the matching protected Review section. Repair, close it, then do the changed retest without copying.

Changed retest if assigned: Classify a shipment as `escalate` only if delayed_days $\geq$ 2 AND priority_customer is True. Test 1/2/3 days and both customer flags.

10. Evidence / mastery

Rule sentence + code + boundary-case table/output + explain-back.

Mastery: changed task passes validation + explain-back. Work corrected with Review open is repair evidence, not mastery.

Practice files: M04_PYTHON_PRACTICE_CANONICAL.py

DE04-05 - Lists, tuples, sets and dictionaries

Source route: Luke Barousse - Python for Data Analytics | 1:34:24-2:12:26 (Lists, dictionaries, sets, tuples) | [OPEN SOURCE](#)
Independent web recheck 2026-09-10: chapter timestamps confirmed.

Starting state / prerequisite: DE04-04.

1. Outcome

Choose list, tuple, set or dict based on order, mutability, uniqueness/membership and key-value lookup needs.

2. Explain - beginner-first model

A list is an ordered mutable sequence. A tuple is an ordered sequence commonly used for a fixed-size record or values that should not be casually changed. A set stores unique hashable values and is excellent for membership/duplicate detection. A dictionary maps unique keys to values.

The important skill is not memorizing brackets; it is selecting a structure that matches the requirement. If the requirement says "unique IDs" or repeated membership tests, a set is a natural fit. If it says "manager for each region", a dictionary expresses that relationship directly.

3. Demonstrate / inspect

```
pipeline = ["extract", "transform", "load"]
coords = (23.3, 85.3)
ids = {"01", "02", "01"}
manager = {"East": "Asha", "West": "Rohit"}
print(pipeline, coords, ids, manager["East"])
```

4. Guided reproduction

Build one list, tuple, set and dict from a small operations example. For each, say what property made it appropriate.

5. Validate the guided result

Choices match the requirement; duplicates in the set collapse intentionally; dictionary keys are meaningful/unique.

6. Recover when stuck

Ask four questions: do I need order? mutation? unique membership? key-to-value lookup? Choose the simplest structure that answers the requirement.

7. Independent task - lesson/video closed

Map three regions to managers with a dict; deduplicate order IDs with a set; keep a three-step pipeline in a list; store one fixed coordinate pair in a tuple.

8. Explain in your own words

Give one realistic case where a set is better than a list and one where a dict is better than both.

9. Protected review / fresh retry rule

Save the genuine independent attempt first. If weak, open only the matching protected Review section. Repair, close it, then do the changed retest without copying.

Changed retest if assigned: Represent warehouse zones and supervisors, a unique set of failed batch IDs, a processing sequence, and a fixed `(year, month)` period.

10. Evidence / mastery

Four structures + printed behavior + structure-choice explanation.

Mastery: changed task passes validation + explain-back. Work corrected with Review open is repair evidence, not mastery.

Practice files: M04_PYTHON_PRACTICE_CANONICAL.py

DE04-06 - Loops and iteration

Source route: Luke Barousse - Python for Data Analytics | 2:21:06-2:44:16 (Loops and list-comprehension lead-in) | [OPEN SOURCE](#)
Independent web recheck 2026-09-10: loop chapter confirmed.

Starting state / prerequisite: DE04-05.

1. Outcome

Iterate through collections, maintain explicit accumulators/counters and validate a loop with an independent control.

2. Explain - beginner-first model

A `for` loop visits items from an iterable one at a time. State that changes across iterations - such as a running total, count or output list - should be initialized before the loop and updated at the correct indentation level.

While learning, a clear loop is often better than compressing everything into one line. After the loop works, compare against an independent check such as `sum(values)` so you know the accumulator did what you intended.

3. Demonstrate / inspect

```
values = [1700, 1350, 1100]
total = 0
for value in values:
    total += value
print(total)
```

4. Guided reproduction

Total a list manually, count values ≥ 1500 and collect values below 1000. Only after the loop, compare the total with `sum()`.

5. Validate the guided result

Each input is processed once; accumulator starts at the correct value; control total matches; output list contains only eligible values.

6. Recover when stuck

Print the accumulator inside the loop to see each step. Check indentation and initialization before changing the algorithm.

7. Independent task - lesson/video closed

From [1700,1350,1100,850,1800], compute total, count ≥ 1500 , and collect values below 1000 using a loop.

8. Explain in your own words

What state changes on each iteration, and how did you independently validate the final state?

9. Protected review / fresh retry rule

Save the genuine independent attempt first. If weak, open only the matching protected Review section. Repair, close it, then do the changed retest without copying.

Changed retest if assigned: From [925, 2100, 1750, 600, 2400, 1100], calculate total, count ≥ 1800 and collect values under 1000.

10. Evidence / mastery

Loop code + step/output evidence + independent total check.

Mastery: changed task passes validation + explain-back. Work corrected with Review open is repair evidence, not mastery.

Practice files: M04_PYTHON_PRACTICE_CANONICAL.py

DE04-07 - Functions and scope

Source route: Luke Barousse - Python for Data Analytics | 2:52:12-3:12:46 (Functions; lambda is optional context) | [OPEN SOURCE](#)
Independent web recheck 2026-09-10: functions chapter confirmed.

Starting state / prerequisite: DE04-06.

1. Outcome

Write small reusable functions with explicit parameters, returned values, predictable behavior and no accidental global dependencies.

2. Explain - beginner-first model

A function gives a name to reusable logic. Parameters are inputs supplied by the caller; local variables exist inside the function; `return` sends a value back to the caller. `print` only displays something and is not a substitute for returning data needed by later code.

Prefer functions whose result depends on their explicit inputs. Hidden globals make testing and reuse harder because a caller cannot see all dependencies from the function signature.

3. Demonstrate / inspect

```
def gross_sales(qty, unit_price):  
    return qty * unit_price
```

```
print(gross_sales(2, 850))  
print(gross_sales(3, 450))
```

4. Guided reproduction

Write two small functions, call each with at least two input sets, and show the difference between a function that prints and one that returns.

5. Validate the guided result

Different inputs produce expected results; returned value can be assigned/reused; no global variable is needed for the calculation.

6. Recover when stuck

Check three things separately: parameters, calculation, and return path. Temporarily print parameters at the function boundary if the wrong value is entering.

7. Independent task - lesson/video closed

Write `summarize\_order(order\_id, qty, unit\_price)` returning a dict with `order\_id` and `gross\_sales`. Test two inputs.

8. Explain in your own words

Why is `return` important for data pipelines, and what risk does hidden global state create?

9. Protected review / fresh retry rule

Save the genuine independent attempt first. If weak, open only the matching protected Review section. Repair, close it, then do the changed retest without copying.

Changed retest if assigned: Write `shipment\_cost(shipment\_id, cartons, cost\_per\_carton)` returning a dict. Test three inputs including zero cartons.

10. Evidence / mastery

Function code + at least two calls + reusable return proof + explain-back.

Mastery: changed task passes validation + explain-back. Work corrected with Review open is repair evidence, not mastery.

Practice files: M04_PYTHON_PRACTICE_CANONICAL.py

DE04-08 - Modules, imports and project structure

Source route: Luke Barousse - Python for Data Analytics | 3:12:46-3:40:05 (Modules and Python library lead-in) | [OPEN SOURCE](#)
Independent web recheck 2026-09-10: modules chapter confirmed.

Starting state / prerequisite: DE04-07. Local folder-based Python becomes required here if you used only a browser notebook earlier.

1. Outcome

Split reusable helpers from an entry script, import them from the intended project root, and explain why folder/run location affects imports.

2. Explain - beginner-first model

A Python module is a `.py` file. Importing lets one file use names defined in another file instead of duplicating code. A small project can have an entry script plus helper module; later modules will formalize larger package structure.

Imports depend on where Python can find modules. For this course, keep the supplied files together, run commands from the intended project folder, and avoid random ``sys.path`` hacks. This is also the first point where a normal local project folder is required for the default route.

3. Demonstrate / inspect

```
# stage04_helpers.py
def gross_sales(qty, unit_price):
    return qty * unit_price

# M04_MODULE_DEMO_CANONICAL.py
from stage04_helpers import gross_sales
print(gross_sales(2, 850))
```

4. Guided reproduction

Complete the two TODO helpers in ``stage04_helpers.py``, then run ``M04_MODULE_DEMO_CANONICAL.py`` from the practice-files folder.

5. Validate the guided result

Import succeeds from the documented folder; helper logic lives only in the helper module; running the entry file prints expected results.

6. Recover when stuck

Confirm filenames and current working directory first. Run from the folder containing both files. Do not rename files halfway through without updating imports.

7. Independent task - lesson/video closed

Create ``helpers.py`` with one reusable calculation and a separate ``run_demo.py`` that imports it. Prove a clean run after closing/reopening the terminal/editor.

8. Explain in your own words

What is a module, what does import do, and why can the same import fail from the wrong project/run location?

9. Protected review / fresh retry rule

Save the genuine independent attempt first. If weak, open only the matching protected Review section. Repair, close it, then do the changed retest without copying.

Changed retest if assigned: Create a fresh ``shipment_helpers.py`` plus ``run_shipment_demo.py`` with different function names/inputs; do not copy your first file names.

10. Evidence / mastery

Two-file project + clean-run command/output + brief module/import explanation.

Mastery: changed task passes validation + explain-back. Work corrected with Review open is repair evidence, not mastery.

Practice files: `stage04_helpers.py`, `M04_MODULE_DEMO_CANONICAL.py`

DE04-09 - Exceptions and debugging basics

Source route: Corey Schafer - Using Try/Except Blocks for Error Handling | Full focused lesson | [OPEN SOURCE](#)
Independent URL/title recheck 2026-09-10. Teaching Book controls modern course behavior.

Starting state / prerequisite: DE04-08.

1. Outcome

Read a traceback, isolate one defect at a time, and handle only expected/recoverable exceptions without hiding unrelated failures.

2. Explain - beginner-first model

A traceback shows the path of calls leading to an exception. Start at the final exception type/message, then find the first relevant line in your code. Fix one root cause and rerun so you know which change affected the behavior.

Exception handling is not a way to make errors disappear. Catch only errors you can handle meaningfully, such as a `ValueError` when converting user text to an integer. A broad `except: pass` can silently turn bad data into missing output, which is dangerous in data engineering.

3. Demonstrate / inspect

```
raw = "12.5"
try:
    qty = int(raw)
except ValueError:
    print(f"Invalid integer quantity: {raw!r}")
```

4. Guided reproduction

Run `M04\_DEBUG\_ME\_CANONICAL.py` unchanged. Save the first traceback. Repair one problem, rerun, and repeat until valid input works and invalid/missing values fail visibly.

5. Validate the guided result

First error is preserved; each fix is isolated; expected bad input has a useful message; no bare/broad swallowing of exceptions.

6. Recover when stuck

If you are changing many lines at once, restore the file and work one traceback at a time. If unsure what to catch, first let the error occur and identify its specific exception type.

7. Independent task - lesson/video closed

Write a `parse\_quantity(raw)` function that returns an int for valid text and raises or reports a clear problem for invalid text without hiding unrelated exceptions.

8. Explain in your own words

Why is swallowing every exception dangerous in a data pipeline?

9. Protected review / fresh retry rule

Save the genuine independent attempt first. If weak, open only the matching protected Review section. Repair, close it, then do the changed retest without copying.

Changed retest if assigned: Write `parse\_unit\_cost(raw)` accepting numeric text such as "425.50" and rejecting "unknown" with a specific, visible error path.

10. Evidence / mastery

Original traceback + repaired code/output + one deliberate bad-input proof + explanation.

Mastery: changed task passes validation + explain-back. Work corrected with Review open is repair evidence, not mastery.

Practice files: M04_DEBUG_ME_CANONICAL.py

DE04-10 - Files, paths, CSV and JSON I/O

Source route: Corey Schafer - File Objects: Reading and Writing to Files | Full focused file-I/O lesson; book supplies pathlib, CSV and JSON specifics | [OPEN SOURCE](#)
Independent web recheck 2026-09-10: video/title confirmed.

Starting state / prerequisite: DE04-09.

1. Outcome

Build file paths predictably, read/write text and structured JSON, and inspect CSV/JSON without confusing file format with business grain.

2. Explain - beginner-first model

``pathlib.Path`` represents filesystem paths without manual slash juggling. Build course paths from the project/script location when you need repeatability. Text files should use an explicit encoding such as UTF-8.

CSV represents records as delimited rows; JSON can represent dictionaries, lists and nested structures. Reading a file successfully does not prove its contents are correct: validate expected file, row/key structure and control values after reading. Avoid overwriting raw course input files.

3. Demonstrate / inspect

```
from pathlib import Path
import json
root = Path(__file__).parent
out = root / "run_manifest.json"
out.write_text(json.dumps({"run_id": "R1", "rows": 3}, indent=2), encoding="utf-8")
loaded = json.loads(out.read_text(encoding="utf-8"))
assert loaded["rows"] == 3
```

4. Guided reproduction

Use `pathlib` to locate the packaged retail CSV. Write a small JSON run manifest, read it back, and compare each expected key/value.

5. Validate the guided result

`Path` does not depend on a random working folder; UTF-8 is explicit; input raw file remains unchanged; JSON read-back matches what was written.

6. Recover when stuck

Print the resolved path before assuming the file is missing. If JSON parsing fails, inspect the exact file text and error location rather than rewriting it blindly.

7. Independent task - lesson/video closed

Write JSON containing source filename, expected row count and status; reload and validate all three values. Also read the CSV header with the standard ``csv`` module.

8. Explain in your own words

What problem does `pathlib` solve, and why should you read back a generated file before trusting it?

9. Protected review / fresh retry rule

Save the genuine independent attempt first. If weak, open only the matching protected Review section. Repair, close it, then do the changed retest without copying.

Changed retest if assigned: Create a fresh manifest for ``M04_RETEST_SHIPMENTS_CANONICAL.csv``, then read its CSV header and validate the required fields without `pandas`.

10. Evidence / mastery

Script + generated JSON + read-back assertions/output + resolved input path.

Mastery: changed task passes validation + explain-back. Work corrected with Review open is repair evidence, not mastery.

Practice files: M04_JSON_IO_SAMPLE.json, M04_RETAIL_ORDERS_CANONICAL.csv, M04_RETEST_SHIPMENTS_CANONICAL.csv

DE04-11 - pandas foundations: load, inspect, filter and aggregate

Source route: Luke Barousse - Python for Data Analytics | 4:22:56-5:08:26 (Pandas intro, inspection, cleaning, analysis) | [OPEN SOURCE](#)

Independent web recheck 2026-09-10: pandas chapter map confirmed. pandas current release check: 3.0.5 on 2026-09-10.

Starting state / prerequisite: DE04-10. Install pandas only now, at its first required use.

1. Outcome

Load a CSV into pandas, inspect shape/types, filter rows deliberately, calculate a column, aggregate at a stated grain and reconcile totals.

2. Explain - beginner-first model

A pandas DataFrame is a table-like in-memory object. Start by inspecting shape, columns, sample rows and dtypes; do not immediately transform unfamiliar data. A boolean filter returns only rows meeting a condition. `groupby` changes the result grain, so say what one output row represents before interpreting totals.

For the course dataset, calculate `sales = qty \* unit\_price`, then filter `status == "Completed"`. Reconcile the grouped regional sales total to a separate sum over the same completed rows. That independent total is a control against accidental omission or double counting.

3. Demonstrate / inspect

```
import pandas as pd
df = pd.read_csv("M04_RETAIL_ORDERS_CANONICAL.csv")
df["sales"] = df["qty"] * df["unit_price"]
completed = df.loc[df["status"].eq("Completed")].copy()
by_region = completed.groupby("region", as_index=False)["sales"].sum()
assert by_region["sales"].sum() == completed["sales"].sum()
```

4. Guided reproduction

Inspect shape/columns/dtypes/head. Calculate sales. Filter Completed. Group by region. Save row counts and reconcile grouped sum to a separate Completed total.

5. Validate the guided result

Canonical controls: input rows=25; Completed rows=23; Completed sales=39320.00; missing sales_rep=1. Regional Completed totals: East 11700, North 9310, South 10210, West 8100.

6. Recover when stuck

If totals differ, compare in stages: input row count -> filter count -> calculated sales -> grouped total. Do not add `dropna` or `fillna` merely to make numbers match.

7. Independent task - lesson/video closed

Return Completed sales by region, top 3 Completed orders by sales, and rows with missing sales_rep. Write the result grain for each output.

8. Explain in your own words

Why can a DataFrame look correct while still answering the wrong business question?

9. Protected review / fresh retry rule

Save the genuine independent attempt first. If weak, open only the matching protected Review section. Repair, close it, then do the changed retest without copying.

Changed retest if assigned: Using the shipments retest CSV, calculate shipment_value, keep Delivered only, aggregate by hub, list top 3 Delivered shipments and count missing coordinator values. Use the separate retest controls only after your query is complete.

10. Evidence / mastery

Code + shape/dtype note + filter count + group result + independent reconciliation + output-grain statements.

Mastery: changed task passes validation + explain-back. Work corrected with Review open is repair evidence, not mastery.

Practice files: M04_RETAIL_ORDERS_CANONICAL.csv, M04_RETEST_SHIPMENTS_CANONICAL.csv

DE04-12 - Practical Big-O intuition

Source route: freeCodeCamp/Jovian - Data Structures and Algorithms in Python | 0:50:52 (Complexity and Big O notation) | [OPEN SOURCE](#)

Independent chapter-list recheck 2026-09-10: Big-O timestamp confirmed.

Starting state / prerequisite: DE04-11.

1. Outcome

Recognize practical $O(1)$, $O(n)$ and $O(n^2)$ growth patterns and choose obviously better membership/counting structures without claiming exact runtime.

2. Explain - beginner-first model

Big-O describes how resource use grows as input size grows; it is not a stopwatch prediction. A single scan over n records is $O(n)$. A nested full scan can become $O(n^2)$. Dictionary/set lookup is average $O(1)$, while scanning a list for each membership check is $O(n)$ per lookup.

In data engineering, this intuition helps you notice code that repeatedly scans a growing collection. You do not need competitive-programming depth here; the target is to recognize obviously poor patterns and choose a suitable built-in structure.

3. Demonstrate / inspect

```
seen = set()
for order_id in ["01", "02", "02"]:
    if order_id in seen:
        print("duplicate", order_id)
    seen.add(order_id)
```

4. Guided reproduction

Compare repeated membership checks against a list and a set. Describe expected growth rather than timing one tiny example and declaring a universal winner.

5. Validate the guided result

Explanation correctly distinguishes list scan from average set lookup and labels a simple nested full scan as $O(n^2)$.

6. Recover when stuck

Count how many times the inner operation may run as n grows. Ignore small constant factors at this conceptual level.

7. Independent task - lesson/video closed

Explain why a set helps repeated duplicate-ID checks; write one nested-loop $O(n^2)$ example you would avoid for large data.

8. Explain in your own words

What does Big-O tell you, and what does it not tell you?

9. Protected review / fresh retry rule

Save the genuine independent attempt first. If weak, open only the matching protected Review section. Repair, close it, then do the changed retest without copying.

Changed retest if assigned: Compare "for each shipment, scan every known failed ID" with building a failed-ID set once. Explain the growth of both approaches.

10. Evidence / mastery

Short code/example + growth explanation + one DE-oriented choice.

Mastery: changed task passes validation + explain-back. Work corrected with Review open is repair evidence, not mastery.

Practice files: M04_CODING_DRILLS_CANONICAL.py

DE04-13 - Coding drills: lists and strings

Source route: Course-built independent coding drills | No external video by design | Course-built: no external dependency.

Starting state / prerequisite: DE04-12. Keep answer keys/AI solving closed during the genuine attempt.

1. Outcome

Solve small list/string problems from a blank function body, state edge cases and prove results with tests.

2. Explain - beginner-first model

Coding fluency grows by translating a requirement into input/output examples, a simple plan, code and tests. For these drills, the absence of a tutorial video is intentional. First write one or two examples and the plain-English steps you expect the function to perform.

String/list operations from earlier lessons are enough. Avoid importing a library that solves the entire exercise unless the requirement allows it, because the objective is reasoning fluency rather than package discovery.

3. Demonstration boundary

No finished drill solution is shown before your saved attempt. Use the visible test file to understand the contract without revealing the implementation.

4. Guided reproduction

Warm-up: write a plain-English algorithm for reversing word order while deciding how to treat extra spaces. Then implement only after the plan is saved.

5. Validate the guided result

Run ``M04_CODING_DRILLS_VISIBLE_TESTS.py``; supplied tests pass and you add at least one edge case of your own.

6. Recover when stuck

Shrink the failing input. Print intermediate values. Return to the input/output contract before changing several lines at once.

7. Independent task - lesson/video closed

Complete ``reverse_words`` and ``count_vowels`` in ``M04_CODING_DRILLS_CANONICAL.py``, then add one edge-case test for each.

8. Explain in your own words

Describe one drill from input -> algorithm -> output -> edge case without reading your code.

9. Protected review / fresh retry rule

Save the genuine independent attempt first. If weak, open only the matching protected Review section. Repair, close it, then do the changed retest without copying.

Changed retest if assigned: In ``M04_FRESH_RETEST_TODO.py``, implement a changed ``normalize_words(text)`` task that strips extra spaces, lowercases tokens and returns them as a list. Add tests for blank and multi-space text.

10. Evidence / mastery

Saved first attempt + test output + invented edge cases + explain-back.

Mastery: changed task passes validation + explain-back. Work corrected with Review open is repair evidence, not mastery.

Practice files: `M04_CODING_DRILLS_CANONICAL.py`, `M04_CODING_DRILLS_VISIBLE_TESTS.py`

DE04-14 - Coding drills: dictionaries and sets

Source route: Course-built independent coding drills | No external video by design | Course-built: no external dependency.

Starting state / prerequisite: DE04-13.

1. Outcome

Use dictionaries for counting/aggregation and sets for uniqueness/membership while preserving deterministic requirements.

2. Explain - beginner-first model

A dictionary is a natural accumulator when each key needs a count or total. `counts[key] = counts.get(key, 0) + 1` is a common pattern. A set is natural when the question is only whether a value has been seen or which unique values exist.

Be careful with "duplicate" requirements: "first value encountered twice" is different from "all unique duplicated values". Define the output contract before coding so repeated duplicates are not emitted accidentally.

3. Demonstrate / inspect

```
regions = ["East", "West", "East"]
counts = {}
for region in regions:
    counts[region] = counts.get(region, 0) + 1
print(counts)
```

4. Guided reproduction

Count three region labels with a dict and detect a repeated ID with a set. Explain why a nested loop is unnecessary.

5. Validate the guided result

Visible tests pass; output contract is deterministic; repeated duplicates are handled exactly as stated.

6. Recover when stuck

Write the expected output for a tiny list by hand. If you need a count/total per key, use a dict accumulator; if only membership matters, start with a set.

7. Independent task - lesson/video closed

Complete `word_counts`, `first_duplicate` and `sales_by_region` in the drill file, then run visible tests and add one edge case each.

8. Explain in your own words

When does a dict hold information that a set cannot?

9. Protected review / fresh retry rule

Save the genuine independent attempt first. If weak, open only the matching protected Review section. Repair, close it, then do the changed retest without copying.

Changed retest if assigned: Implement `count_statuses` and `unique_repeated_batch_ids` on changed warehouse inputs inside the fresh-retest workspace.

10. Evidence / mastery

First attempt + passing visible tests + edge-case tests + data-structure rationale.

Mastery: changed task passes validation + explain-back. Work corrected with Review open is repair evidence, not mastery.

Practice files: M04_CODING_DRILLS_CANONICAL.py, M04_CODING_DRILLS_VISIBLE_TESTS.py

DE04-15 - Stack, queue, search and sort basics

Source route: freeCodeCamp/Jovian - Data Structures and Algorithms in Python | 1:24:57 (Binary vs linear search) + 5:16:47 (Sorting algorithms section); book adds lightweight stack/queue | [OPEN SOURCE](#)
Independent chapter-list recheck 2026-09-10: both mapped timestamps confirmed.

Starting state / prerequisite: DE04-14.

1. Outcome

Recognize LIFO/FIFO behavior, implement linear search, explain binary-search sorted-data precondition and sort without accidentally mutating an input when preservation is required.

2. Explain - beginner-first model

A stack removes the most recently added item first (LIFO). A queue removes the earliest added item first (FIFO); ``collections.deque`` supports efficient appends/pops from both ends. Linear search checks items until it finds a target or reaches the end. Binary search repeatedly halves a sorted search space, so the sorted-data precondition is essential.

``sorted(values)`` returns a new list, while ``values.sort()`` mutates the list. In data work, preserving input order may matter, so choose deliberately.

3. Demonstrate / inspect

```
from collections import deque
stack = []
stack.append("extract"); stack.append("transform")
print(stack.pop())
q = deque(["job1", "job2"]); print(q.popleft())
values = [7,2,9,1,5]; ordered = sorted(values)
print(values, ordered)
```

4. Guided reproduction

Use a list as stack, deque as queue, run a linear-search function and compare ``sorted`` vs ``sort()`` behavior.

5. Validate the guided result

LIFO/FIFO outputs are correct; original list is preserved when required; linear search returns index/-1; binary search precondition is stated.

6. Recover when stuck

Act out the structure with 3 labels. For search, write what happens to each candidate position. For sorting, print both original and result.

7. Independent task - lesson/video closed

Push extract/transform/load and pop once; create a FIFO queue example; sort `[7,2,9,1,5]` without mutating original; implement ``linear_search``.

8. Explain in your own words

Why is binary search invalid on arbitrary unsorted input, and when does ``sorted()`` protect you from accidental mutation?

9. Protected review / fresh retry rule

Save the genuine independent attempt first. If weak, open only the matching protected Review section. Repair, close it, then do the changed retest without copying.

Changed retest if assigned: Use tasks `["ingest","validate","publish"]` for stack/queue contrasts and search target 12 inside `[4,12,7,3]`. Sort a copy only.

10. Evidence / mastery

Stack/queue/search/sort code + output + 60-second explanation.

Mastery: changed task passes validation + explain-back. Work corrected with Review open is repair evidence, not mastery.

Practice files: M04_CODING_DRILLS_CANONICAL.py

DE04-16 - Integrated Python checkpoint and reproducible script habits

Source route: Python Packaging User Guide - Packaging Python Projects | README.md, pyproject.toml, src/ and tests/ structure; only after the saved first checkpoint attempt | [OPEN SOURCE](#)
Official PyPA tutorial independently rechecked 2026-09-10.

Starting state / prerequisite: DE04-01..15 evidence complete. Keep the PyPA reference closed until the first checkpoint attempt is saved.

1. Outcome

Combine core Python skills into a repeatable folder-based program with explicit input validation, deterministic output, visible failure behavior and a documented run command.

2. Explain - beginner-first model

A reproducible script can be run again from known inputs and produce explainable outputs without hidden notebook state. Keep inputs explicit, split logic into small functions, validate required fields before calculating, fail visibly when the input contract is broken, and write deterministic output.

Project structure communicates how to rerun the work. For this checkpoint, keep a small `README.md`, entry/helper scripts, tests/checks and stable input/output locations. PyPA shows the modern `pyproject.toml`/`src`/`tests` convention; packaging/publishing are out of scope, and M06 adds Git/Linux workflow.

3. Demonstration boundary

FIRST ATTEMPT RULE: do not inspect a finished checkpoint solution. Build from `M04\_CHECKPOINT\_TODO.py`, save the first attempt, then use Review R16/PyPA structure guidance only for targeted repair.

4. Guided reproduction

No answer walkthrough. Read the TODO/rubric, state the input/output contract, then implement one function at a time.

5. Validate the guided result

Clean run succeeds; controls reconcile; bad/missing input fails visibly; raw input is unchanged; output is deterministic; documented run command works from a fresh terminal.

6. Recover when stuck

Split failures by boundary: path -> parse -> required columns -> calculation/filter -> aggregation -> output write -> rerun. Repair one boundary at a time.

7. Independent task - lesson/video closed

Complete the checkpoint: load/validate retail CSV, calculate sales, keep Completed, produce regional totals + top 3 + missing-rep count, write deterministic JSON, prove one bad-input failure, and document the run command.

8. Explain in your own words

Explain the program from input contract to validation to output, including one failure mode and how a second person can reproduce it.

9. Protected review / fresh retry rule

Save the genuine independent attempt first. If weak, open only the matching protected Review section. Repair, close it, then do the changed retest without copying.

Changed retest if assigned: If assigned, complete `M04\_FRESH\_CHECKPOINT\_TODO.py` on the changed shipments fixture with R16 closed; preserve the original attempt.

10. Evidence / mastery

First attempt + controls + failure proof + JSON output + run transcript + explain-back.

Mastery: changed task passes validation + explain-back. Work corrected with Review open is repair evidence, not mastery.

Practice files: M04_CHECKPOINT_TODO.py, M04_RETAIL_ORDERS_CANONICAL.csv, M04_FRESH_CHECKPOINT_TODO.py, M04_RETEST_SHIPMENTS_CANONICAL.csv